

leave p out cv

Leave-p-out Cross-Validation (LpOCV) is a validation method where you:

👉 **Leave out p samples at a time** as the test set, train on the rest, and repeat this for *all possible combinations* of size p .

It's a generalization of Leave-One-Out CV.

◆ How it works

Given a dataset with **N samples**:

1. Choose a number **p** (e.g., 2, 3, 5...).
2. Generate **all possible subsets of size p** from the dataset.
3. For each subset:
 - Use those **p samples as the test set**
 - Use the remaining **$N - p$ samples as the training set**
4. Train → Test → Record error
5. Final score = **average of all test errors**

◆ Example

Suppose:

Dataset = [A, B, C, D] ($N = 4$), $p = 2$

All possible test sets:

Iteration	Test Set	Train Set
1	[A, B]	[C, D]
2	[A, C]	[B, D]
3	[A, D]	[B, C]

Iteration	Test Set	Train Set
4	[B, C]	[A, D]
5	[B, D]	[A, C]
6	[C, D]	[A, B]

➡ Total models trained = $C(4,2) = 6$

◆ Relation to Other CV Methods

Method	Special Case
Leave-One-Out CV	$p = 1$
Leave-p-Out CV	$p \geq 1$
k-Fold CV	Random partitions (not all combinations)

◆ Pros

- ✓ Uses **all possible splits** → very accurate
- ✓ Low bias
- ✓ Good for **very small datasets**

◆ Cons

- ✗ **Extremely expensive** computationally
- ✗ Number of models = $C(N, p)$ grows fast
- ✗ Not practical for large N

◆ When to use LpOCV

- ✓ Dataset is **very small**
- ✓ You want **maximum evaluation precision**
- ✓ Computation cost is not a concern

◆ Formula for number of models

$$\text{Total runs} = \binom{N}{p} = \frac{N!}{p!(N-p)!}$$

If you want, next I can explain:

- LpOCV vs k-Fold with a graph 📊
- LpOCV with Iris dataset example 🌸
- Python / Sklearn implementation 🔧



leave one cv

Leave-One-Out Cross-Validation (LOOCV) is a special case of Leave-p-Out CV where:

👉 $p = 1 \rightarrow$ you leave out **one sample at a time** as the test set.

◆ How LOOCV works

For a dataset with **N samples**:

1. Pick **1 sample** → use it as the **test set**
2. Use the remaining **N-1 samples** as the **training set**
3. Train the model
4. Test on the left-out sample
5. Repeat this for **every sample**
6. Final performance = **average of all N test errors**

➡ Total models trained = **N**

◆ Example

Dataset = [A, B, C, D] (N = 4)

Iteration	Test Sample	Train Samples
1	A	B, C, D
2	B	A, C, D
3	C	A, B, D
4	D	A, B, C

◆ Key Properties

Property	LOOCV
Test size	1 sample
Train size	N-1
Models trained	N
Bias	Low
Variance	High
Cost	Expensive for large N

◆ Pros

- ✓ Uses **almost all data** for training each time
- ✓ Good for **small datasets**
- ✓ Low bias in performance estimate

◆ Cons

- ✗ **High variance** (test set is just 1 point)
 - ✗ Computationally heavy for big datasets
 - ✗ Not suitable for large N
-

◆ When to use LOOCV

- ✓ Dataset is **small**
 - ✓ You want to use **maximum training data**
 - ✓ Model is **fast to train**
-

◆ Python (sklearn) example

python

```
from sklearn.model_selection import LeaveOneOut, cross_val_score
from sklearn.linear_model import LogisticRegression
from sklearn.datasets import load_iris

X, y = load_iris(return_X_y=True)

loo = LeaveOneOut()
model = LogisticRegression(max_iter=200)

scores = cross_val_score(model, X, y, cv=loo)

print("LOOCV Accuracy:", scores.mean())
```

If you'd like, I can next explain:

- LOOCV vs k-Fold CV
- LOOCV with graphs 📊
- Bias-Variance tradeoff in CV

📄 Copy code



k-Fold CV

 k-Fold Cross-Validation (k-Fold CV)

k-Fold Cross-Validation is a resampling technique used to evaluate machine learning models by:

👉 Splitting the dataset into **k equal parts (folds)**, training on **k-1 folds**, and testing on the **remaining fold** — repeated **k times**.

 How it works

1. Choose a number **k** (common: 5 or 10)
 2. Split the data into **k folds**
 3. For each iteration $i = 1$ to k :
 - Use fold i as **test set**
 - Use remaining $k-1$ folds as **training set**
 4. Train → Test → Save score
 5. Final result = **Average of all k scores**
-

 Example (k = 5)

Dataset = 100 samples → 5 folds of 20 samples each

Iteration	Test Fold	Train Folds
1	Fold 1	2, 3, 4, 5
2	Fold 2	1, 3, 4, 5
3	Fold 3	1, 2, 4, 5
4	Fold 4	1, 2, 3, 5
5	Fold 5	1, 2, 3, 4

◆ Types of k-Fold

Variant	Description
Standard k-Fold	Random splits
Stratified k-Fold	Preserves class ratio (for classification)
Group k-Fold	Keeps same group in one fold
TimeSeriesSplit	For time-based data

◆ Pros

- ✓ Good **bias–variance tradeoff**
- ✓ Works well for **small & medium datasets**
- ✓ More stable than Holdout
- ✓ Efficient vs LOOCV

◆ Cons

- ✗ k times training → more cost
- ✗ Not ideal for time-series unless modified

◆ Choosing k

k value	Behavior
k = 2	High bias
k = 5	Good balance ✓
k = 10	More accurate but slower
k = N	Becomes LOOCV

◆ Python Example (sklearn)

python

 Copy code

```

from sklearn.model_selection import KFold, cross_val_score
from sklearn.linear_model import LogisticRegression
from sklearn.datasets import load_iris

X, y = load_iris(return_X_y=True)

kf = KFold(n_splits=5, shuffle=True, random_state=42)
model = LogisticRegression(max_iter=200)

scores = cross_val_score(model, X, y, cv=kf)

print("k-Fold Accuracy:", scores.mean())

```

◆ Visual intuition

mathematica

 Copy code

```

Data:  [ A B C D E F G H I J ]
Fold 1: [ A B ] -> test
Fold 2: [ C D ] -> test
Fold 3: [ E F ] -> test
Fold 4: [ G H ] -> test
Fold 5: [ I J ] -> test

```

Since you're learning ML concepts step-by-step (Naive Bayes → CV → Regression, etc.), next I can explain:

👉 **Stratified k-Fold with Iris dataset**

≡ ChatGPT  

 ...

 Holdout vs k-Fold vs LOOCV (with graph) 

